

AI Labs Report

Tasks 1–4: Prompt-Driven Resume Classification, Job Market Analytics Dashboard, AI Recruitment Copilot (RAG) and Candidate Shortlisting & Outreach Agent (LangGraph)

Task 1 — Resume & Job Posting Classification via Prompt Engineering

Context

Recruiters and job platforms deal with a huge volume of unstructured resume and job posting text, and manually tagging it by category, seniority, or sentiment is slow and honestly pretty inconsistent from person to person.

Problem

There wasn't a quick, training-free way to sort resume or job-posting text into structured categories (skills domain, seniority level, sentiment toward the role) without building or fine-tuning a custom model.

Approach

I used a labelled dataset of 60 resumes and ran a local Llama model through Ollama (llama3.2:latest) to do three prompt-driven NLP tasks: domain classification, entity extraction (skills / years of experience), and match-suitability summarization. For classification I compared zero-shot vs few-shot prompting directly against each other, and for entity extraction I compared zero-shot vs chain-of-thought.

Results

Here's what actually happened when I ran it against the labelled set:

Metric	Zero-shot	Few-shot
Classification accuracy	15% (9/60)	20% (12/60)
Both correct	8	8
Both wrong	47	47

Few-shot prompting did edge out zero-shot, but honestly both numbers are low, and I don't think that's purely a model failure. When I went through the misclassified rows manually, several of the 'true' labels in the dataset didn't actually match the resume content. For example, one resume was labelled HR & Administration in the dataset but the actual text was a B.Tech graduate describing machine learning job interests — the model classified it as Data & AI, which is arguably the more correct answer even though it was scored wrong. So a decent chunk of the low accuracy is dataset label noise, not the model misunderstanding the text.

Both prompt styles also showed a clear bias toward predicting 'Data & AI' — it was the most common prediction in both zero-shot (35 out of 60) and few-shot (34 out of 60), likely because the dataset itself is skewed toward AI/data-related resumes (15 out of 60 true labels), so the model leaned into that pattern.

Entity extraction (skills, years of experience, education level) worked noticeably better than classification in a qualitative sense — both zero-shot and chain-of-thought reliably pulled out skill lists and education level even when the resume text was short or vague, though years of experience was frequently returned as 'unknown' since most of the sample resumes didn't state it explicitly.

The match-suitability summaries (zero-shot vs role-aware prompting) were the most consistently useful output — both prompt versions produced readable, recruiter-style reasoning, and the role-aware version tended to be more specific about skill gaps rather than just restating the resume.

Takeaway

Prompt-only classification without fine-tuning is workable for narrower tasks like entity extraction and summarization, but for multi-class domain classification, both prompting strategy and — just as much — the quality of the ground-truth labels matter a lot. Few-shot gave a real but modest lift over zero-shot; the bigger lever would probably be cleaning the dataset.

Task 2 — Job Market Analytics Dashboard

Context

Job seekers and HR teams often work off gut-feel impressions of which skills are 'hot,' without a structured, data-backed view of which skills are actually rising or declining across roles or industries.

Problem

There wasn't an easy way to see which skills are trending, where the gaps are between what job postings ask for and what candidates actually have, and how that varies by role or domain — without manually scanning postings one by one.

Approach

I built an ETL pipeline from a resume/job posting dataset into a structured data model (four CSVs: KPI summaries, domain gaps, role breakdowns, and skill gaps), then built an interactive Tableau dashboard with KPI cards, drill-downs, and an anomaly table for skill demand vs supply.

Results — Dashboard KPIs

KPI	Value
Total unique skills demanded	97
Top demanded skill	AutoCAD
Average candidate match rate	1.5%
Biggest skill gap	Human Resource Management
Worst performing domain	Software Engineering

The domain gap breakdown showed IT Infrastructure with by far the largest gap score (8,470), followed by Civil & Mechanical Engineering (6,405) and Business & Marketing (5,706) — meaning these domains have the widest distance between what's demanded and what candidates in the dataset actually supply.

The role breakdown showed AI Engineer and Data Engineer with the highest average match rates among roles that had any match at all, while several roles (Asst. Manager/Manager, Business Development Executive) showed essentially zero average match — candidates applying for or tagged under those roles had almost none of the required skills present in their resumes.

The anomaly table was probably the most eye-opening part. For Business & Marketing specifically, a whole cluster of skills — brand promotion, campaign management, corporate communications, Facebook ads, Facebook campaign management, Google Ads, Google Analytics — all showed a demand count of 339 with a matching gap of 339, meaning supply count was effectively zero. Nobody in the candidate pool had these skills at all, despite consistent demand.

Takeaway

The 1.5% overall match rate is a pretty stark number — it suggests the candidate pool in this dataset, taken as a whole, is a poor fit for what the job postings are actually asking for, especially in domains like Business & Marketing where certain skill clusters (digital marketing/social ads) are basically unrepresented in supply. For an HR analyst, this dashboard would flag digital marketing skills and IT infrastructure skills as the most urgent hiring/training gaps to address.

Task 3 — AI Recruitment Copilot using Retrieval-Augmented Generation (RAG)

Context

Most companies get hundreds of resumes per opening, and the ATS tools they use mostly just scan for keywords instead of actually understanding what's in the resume.

Problem

Because of this keyword-only matching, recruiters can miss good candidates whose resumes use different words than the job description, and the system doesn't explain why it picked or rejected someone.

Approach

I used resume data along with semantic embeddings and RAG so the system compares resumes and job descriptions based on meaning, not just keywords, and pulls out the relevant resume sections to support its decision. Rather than build this as a standalone script, I integrated it directly into EnterpriseBot — an existing Flask chatbot I'd already built for a different task — so the recruitment copilot works through the same chat interface as general conversation, with an intent classifier routing messages to either the general chatbot or the recruitment RAG pipeline.

Tech stack

LLM: Groq (llama-3.3-70b-versatile) for both intent classification and match report generation

Embeddings: sentence-transformers, all-MiniLM-L6-v2

Vector store: FAISS

PDF extraction: pdfplumber, with a pytesseract + pdf2image OCR fallback for scanned/image-based resumes

Backend: Flask (existing EnterpriseBot app)

Build process and issues along the way

This was a genuinely iterative build with several real problems that came up, not just a clean implementation:

- PDF extraction returning empty text: about half of my test resumes (fancier resume-template PDFs downloaded from template sites) turned out to have no real text layer at all — pdfplumber returned an empty string. Traced this to the PDFs being flattened/graphic-based rather than genuine text PDFs, so I added an OCR fallback (pytesseract + pdf2image via Poppler) that only kicks in when normal extraction comes back empty.
- Python 3.9 compatibility: after rebuilding my venv, the RAG module threw a `TypeError` on a type hint (`list[dict] | None`) — that union syntax is Python 3.10+ only, and my environment is on 3.9. Fixed with ``from __future__ import annotations`` at the top of the file rather than rewriting every type hint.

- Missing local tooling: OCR required Homebrew (not installed on my machine) plus Tesseract and Poppler as system-level dependencies, on top of the Python packages.
- Frontend gap: the existing 'New Chat' button in EnterpriseBot's UI was present in the HTML but had no JavaScript wired to it at all — clicking it did nothing. Added an event listener that resets the chat window back to the greeting message.

Results

Once wired up, the pipeline works as intended: a natural-language job description typed into the chat (e.g. "Looking for a candidate with accounting experience, QuickBooks knowledge, and financial reporting skills") gets routed by the intent classifier to the recruitment pipeline, which retrieves the top-3 best-matching candidates across the resume pool (grouped by candidate rather than by raw chunk, so one candidate can't crowd out the top-3 with multiple matching chunks), and returns a per-candidate match score, matching skills, missing skills, and a short reasoning explanation.

In testing, the reasoning held up well to scrutiny — for a QuickBooks-focused job description, it correctly identified candidates with strong accounting/reporting overlap but flagged them as missing QuickBooks specifically, and in one case correctly noted a candidate had experience with a different tool (Zoho Books) instead, rather than just flatly marking the skill as absent.

Takeaway

Semantic retrieval plus an explanation step genuinely solves the stated problem better than keyword ATS — it caught skill overlaps described in different wording and gave a specific, checkable reason for each score rather than a black-box match percentage. The OCR fallback ended up being one of the more important pieces of the whole pipeline, since a meaningful fraction of real-world resumes (especially ones built from flashy templates) aren't actually text-searchable PDFs at all — something a pure-text RAG pipeline would silently fail on without anyone noticing.

Task 4 — Candidate Shortlisting & Outreach Agent (LangGraph)

Context

Recruiters manually orchestrate a multi-step workflow for every open role — screening resumes against a job description, shortlisting the strongest candidates, and drafting personalized outreach to each one — and currently do this by hand for every posting.

Problem

This orchestration is repetitive, slow, and inconsistent between recruiters; there is no system that plans and executes the full screen-to-outreach sequence autonomously while still keeping a human in control of what actually gets sent.

Approach

I will build a LangGraph agent that takes a job description as its goal, retrieves and scores candidates using the RAG pipeline built in Task 3, branches based on whether strong matches exist, drafts personalized outreach emails for shortlisted candidates via Groq, and pauses for human approval before finalizing — with every step logged to an audit trail.

Three test scenarios were run against the live agent to cover a happy path, a conditional branch, and a failure case, as required by the module handbook (§6.6). All runs used the same resume pool (5 PDFs, indexed via the Task 3 RAG pipeline) and the same match-score threshold (50%). Full raw traces for every step are recorded in `audit_log.jsonl`.

Scenario 1 — Happy Path (Strong Match)

Input goal: "accounting"

Expected behavior: Agent retrieves candidates, scores them against the job description, the quality-check node finds at least one strong match (score ≥ 50), routes to the drafting node, drafts personalized outreach emails, pauses for human approval, and finalizes only after explicit approval.

Actual behavior (from audit trail)

Step	Node	Result
1	retrieve_node	Retrieved 3 candidates: id_4, id_3, id_1
2	score_node	Scores: id_4=90, id_3=95, id_1=90
3	quality_check_node	3 strong matches found → branch: strong_match
4	draft_node	3 personalized outreach emails drafted, each referencing the candidate's actual matching skills
5	human_approval_node	Emails presented to human; human responded "approved"
6	finalize_node	Finalized (sandboxed — no real email sent)

Verdict: PASS

The agent correctly identified strong matches, drafted relevant, specific outreach content (not generic filler — each email referenced real skills such as GAAP, financial analysis, or cash flow), and did not finalize anything until explicit human approval was given.

Scenario 2 — Conditional Branch (No Strong Match)

Input goal: "Senior Kubernetes and cloud infrastructure engineer with Go and Terraform experience"

Expected behavior: Agent retrieves the nearest candidates available (FAISS always returns nearest neighbors even if none are good matches), scores them, and — since none of the resumes in the pool have relevant cloud/DevOps skills — the quality-check node should find zero strong matches and route to the no-match path, skipping drafting and human approval entirely.

Actual behavior (from audit trail)

Step	Node	Result
1	retrieve_node	Retrieved 3 candidates: id_5, id_2, id_3
2	score_node	Scores: id_5=0, id_2=0, id_3=0
3	quality_check_node	0 strong matches found → branch: no_match
4	no_match_node	Agent stopped gracefully — no outreach drafted, no human approval requested

Verdict: PASS

The conditional branch worked correctly in both directions. Groq's scoring correctly gave all three candidates a 0% match rather than inflating scores, and the agent did not waste a human approval step on candidates it had already determined were irrelevant.

Scenario 3 — Failure Case (Degenerate Input)

Input goal: "x" (a single, meaningless character used in place of a real job description)

Expected behavior: The agent should fail safely — either by recognizing the input is too degenerate to act on and stopping before drafting outreach, or at minimum by not producing confidently-worded, high-scored outreach based on no real signal.

Actual behavior (from audit trail)

Step	Node	Result
1	retrieve_node	Retrieved 3 candidates: id_3, id_2, id_4 (FAISS returns nearest neighbors regardless of query quality)
2	score_node	Scores: id_3=90, id_2=80, id_4=0
3	quality_check_node	2 strong matches found (90, 80) → branch: strong_match
4	draft_node	2 outreach emails drafted, using generic phrasing ("skilled professional", "your background stands out") since there was no real job requirement to reference
5	human_approval_node	Emails presented; human responded "approved"
6	finalize_node	Finalized (sandboxed — no real email sent)

Verdict: PARTIAL — Structural pass, decision-quality fail

The system did not crash and did not loop indefinitely, which satisfies the handbook's minimum bar for graceful failure handling at the code level. However, this scenario surfaced a real limitation: the agent has no check for whether the input job description is meaningful before proceeding through the full pipeline. Groq's scoring model assigned high-confidence match scores (90%, 80%) to candidates against a one-character input, effectively fabricating relevance rather than recognizing there was no real signal to evaluate. The agent then drafted and requested approval for outreach emails that were, on inspection, generic and not grounded in any actual job requirement.

Recommended fix (not yet implemented)

Add an input-validation guard as the first step in the graph — reject or flag job descriptions under a minimum length/specificity threshold (e.g. under 15-20 characters, or lacking any recognizable skill/role keywords) before the retrieve step runs. This would convert the current silent failure into an explicit, visible one, which is preferable for a system that is meant to support (not replace) human recruiter judgment. This fix was intentionally left undone for this test report so the unguarded failure mode could be documented honestly.

Summary

Scenario	Type	Verdict
1 — "accounting"	Happy path	PASS
2 — Kubernetes/Go/Terraform JD	Conditional branch	PASS
3 — "x"	Failure case	PARTIAL — no crash, but decision-quality limitation identified

Overall, the agent's control flow (retrieval → scoring → conditional branching → drafting → HITL approval → finalization) worked reliably across all three scenarios, including proper graceful termination on both the no-match path and the degenerate-input path. The one substantive limitation found — no upfront validation of input meaningfulness — is a genuine and honestly-reported finding rather than a fabricated edge case, and represents the clearest opportunity for improvement if further development time were available.

